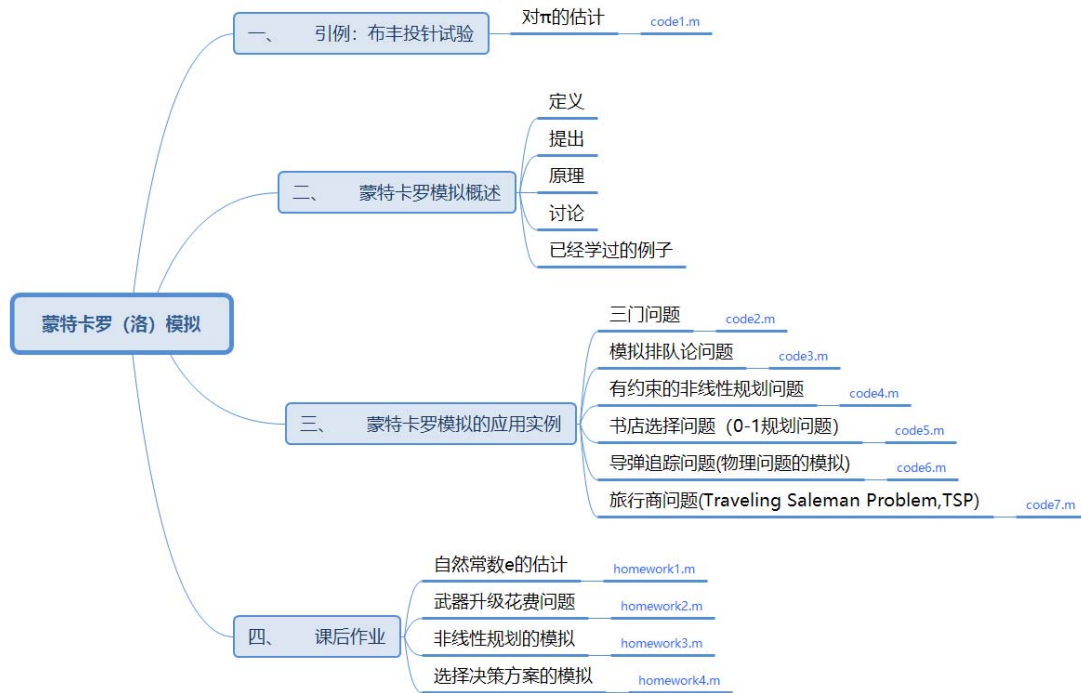
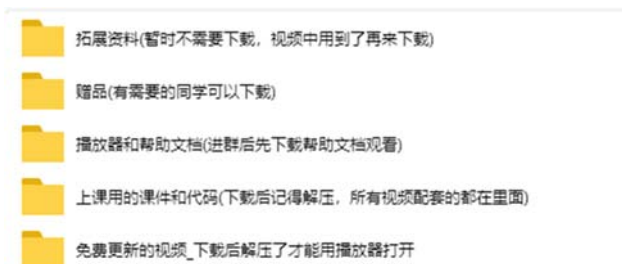


蒙特卡罗模拟



(1) 视频中提到的附件可在[售后群的群文件](#)中下载。

包括讲义、代码、我视频中推荐的资料等。



(2) 关注我的[微信公众号《数学建模学习交流》](#)，后台回复“[软件](#)”两个字，可获得常见的建模软件下载方法；发送“[数据](#)”两个字，可获得建模数据的获取方法；发送“[画图](#)”两个字，可获得数学建模中常见的画图方法。另外，也可以看看公众号的历史文章，里面发布的都是对大家有帮助的技巧。

(3) [购买更多优质精选的数学建模资料](#)，可关注我的微信公众号《数学建模学习交流》，在后台回复“[买](#)”这个字即可进入店铺进行购买。

(4) 视频价格不贵，但价值很高。单人购买观看只需要 **58 元**，和另外两名队友一起购买人均仅需 **46 元**，视频本身也是下载到本地观看的，所以请大家[不要侵犯知识产权](#)，对视频或者资料进行二次销售。

蒙特卡罗(洛)模拟

一. 引例：布丰投针实验

投针步骤

buffon

法国数学家布丰 (1707-1788) 最早设计了投针试验。

(又译布丰)

这一方法的步骤是：

- 1) 取一张白纸，在上面画上许多条间距为 a 的平行线。
- 2) 取一根长度为 l ($l \leq a$) 的针，随机地向画有平行直线的纸上掷 n 次，观察针与直线相交的次数，记为 m 。
- 3) 计算针与直线相交的概率。

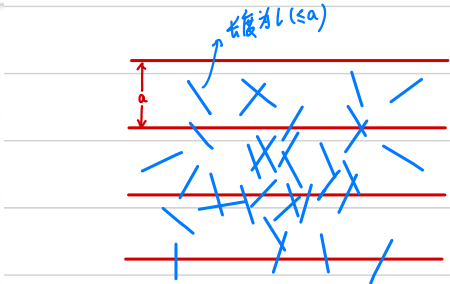
18世纪，法国数学家布丰提出的“投针问题”，记载于布丰1777年出版的著作中：“在平面上画有一组间距为 a 的平行线，将一根长度为 l ($l \leq a$) 的针任意掷在这个平面上，求此针与平行线中任一条相交的概率。”

布丰本人证明了，这个概率是：

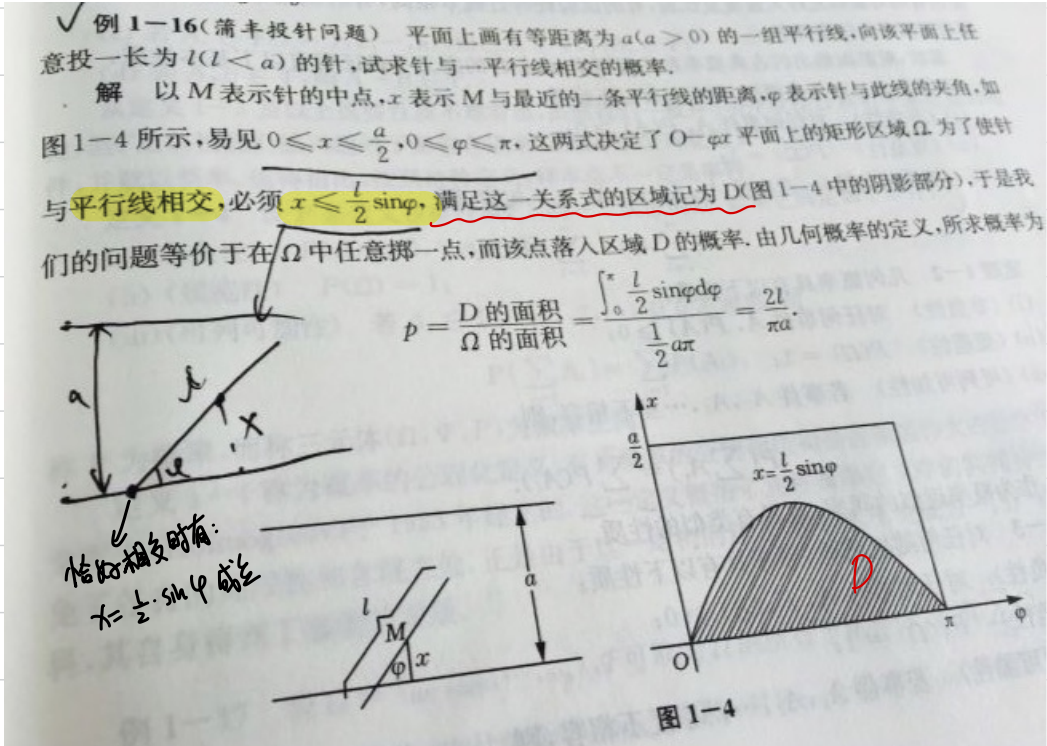
$$p = \frac{2l}{\pi a} \text{ (其中 } \pi \text{ 为圆周率)} \approx \frac{m}{n} \quad \pi = p \cdot a \cdot 2l$$

由于它与 π 有关，于是人们想到利用投针试验来估计圆周率的值。

布丰惊奇地发现：有利的扔出与不利的扔出两者次数的比，是一个包含 π 的表示式。如果针的长度等于 $a/2$ ，那么有利扔出的概率为 $1/\pi$ 。扔的次数越多，由此能求出越精确的 π 的值。



试验者	时间	投掷次数	相交次数	圆周率估计值
Wolf	1850年	5000	2532	3.1596
Smith	1855年	3204	1218.5	3.1554
C.De Morgan	1860年	600	382.5	3.137
Fox	1884年	1030	489	3.1595
Lazzerini	1901年	3408	1808	3.1415929
Reina	1925年	2520	859	3.1795



考研的同学可以看看这道几何概型的题目 (图片来自知乎)

重复多次，由点变为面来求范围 ✓

二. 蒙特卡罗方法概述

$$p = \frac{2L}{\pi a} \approx \frac{m}{n}$$

(1) 定义

蒙特卡罗方法又称统计模拟法，是一种随机模拟方法，以概率和统计理论方法为基础的一种计算方法，是使用随机数（或更常见的伪随机数）来解决很多计算问题的方法。将所求解的问题同一定的概率模型相联系，用电子计算机实现统计模拟或抽样，以获得问题的近似解。为象征性地表明这一方法的概率统计特征，故借用赌城蒙特卡罗命名。

(2) 提出

蒙特卡罗方法于20世纪40年代美国在第二次世界大战中研制原子弹的“曼哈顿计划”计划的成员S.M.乌拉姆和J.冯·诺伊曼首先提出。数学家冯·诺伊曼用驰名世界的赌城—摩纳哥的Monte Carlo—来命名这种方法，为它蒙上了一层神秘色彩。在这之前，蒙特卡罗方法就已经存在。1777年，法国Buffon提出用投针实验的方法求圆周率，这被认为是蒙特卡罗方法的起源。

(3) 原理

由大数定理可知，当样本容量足够大时，事件的发生频率即为其概率。

(4) 讨论

① 蒙特卡罗是一种算法吗？

算法（Algorithm）是指解题方案的准确而完整的描述，是一系列解决问题的清晰指令。蒙特卡罗准确的来说只是一种思想，或者是一种方法。如果我们所求解的问题与概率模型有一定的关联，那么我们就可以使用计算机多次模拟事件发生，以获得问题的近似解。从数学建模角度来看，大家千万别认为蒙特卡罗有一个通用的代码。每个问题对应的代码都是不同的，我们分析清楚题目后，就要自己进行编写适用于这个题目的代码。

② 蒙特卡罗与计算机仿真的关系

抽象 + 编程

计算机仿真（模拟）早期称为蒙特卡罗方法，是一门利用随机数实验求解随机问题的方法，其主要应用在复杂问题的数值模拟上。但随着计算机的性能的提高以及各种新兴产业的发展，目前计算机仿真涉及的内容要广得多，例如过程控制、动画仿真、图像静态模拟等都属于计算机仿真的应用（例如用计算机模拟原子弹爆炸的过程、使用计算机生成特效大片等）。在数学建模中，我们不用刻意的去区分两者的区别，大家只需要知道如何使用计算机对问题进行模拟即可。

③ 蒙特卡罗与枚举法

枚举法是我们中学就接触的算法，就是把所有可能发生情况都考虑进去，最终计算出来一个确定结果。这就与蒙特卡罗方法的思路很类似，蒙特卡罗法模拟的次数越多，计算的就越准确。由于生活中有许多事件发生的结果都有无限种可能（例如一个连续分布的取值），因此我们不可能枚举出所有的结果，这时候就只能通过蒙特卡罗模拟，将一个不确定性的问题转化成很多个确定性问题，并得到一个近似解，因此蒙特卡罗算法也可以看成是枚举法的一种变异。

Np 问题

(5) 已经学过的例子

① 第一期视频(A01)第七讲：多元回归分析

探究内生性对回归结果的影响

更多优质建模资料请关注我的微店店铺“数学建模学习交流”

内生性的蒙特卡罗模拟

假设 $y = 0.5 + 2x_1 + 5x_2 + \mu$, $\mu \sim N(0, 1)$

如果 x_1 在 $[-10, 10]$ 上均匀分布

如果我们用一元线性回归模型: $y = kx_1 + b + \mu$ 进行估计

试探究估计出来的 k 的大小与 $\rho_{x_1, \mu}$ 的关系

相关系数绝对值越大，代表内生性越大

M 数学建模学习交流

更多视频请在Bilibili网站关注UP主：数学建模学习交流

20 / 67

② 第一期视频(A01)番外篇：基于熵权法对TOPS模型的修正

信息熵和标准差的关系

更多优质建模资料请关注我的微店店铺“数学建模学习交流”

熵权法背后的原理

熵权法是一种客观赋权方法

★ 依据的原理：指标的变异程度越小，所反映的信息量也越少，其对应的权值也应该越低。（客观 = 数据本身就可以告诉我们权重）

我们可以用指标的标准差来衡量样本的变异程度，指标的标准差越大，其信息熵越小。

左图是蒙特卡罗的结果
随机生成一组有30个样本且位于区间 [0,1] 上的数据，计算其信息熵和标准差；
将上述步骤重复100次，我们能够得到100组信息熵和标准差的取值，将其绘制成散点图。
可以发现，两个指标之间有很明显的负相关关系。

code_Monte_Carlo.m

M 数学建模学习交流

更多视频请在Bilibili网站关注UP主：数学建模学习交流

9 / 15

三.蒙特卡罗方法的应用实例

(1) 三门问题

你参加一档电视节目，节目组提供了ABC三扇门，主持人告诉你，其中一扇门后边有辆汽车，其它两扇门后是空的。

假如你选择了B门，这时，主持人打开了C门，让你看到C门后什么都没有，然后问你要不要改选A门？

(关于三门问题的讨论可看B站和知乎，我们这里使用蒙特卡罗来进行模拟)



百科 讨论 精华

71 个讨论

若羽

蒙提霍尔问题（又称三门问题、山羊汽车问题）的正解是什么？

(蒙提·霍尔问题,也称三门问题)这是美国有奖游戏节目中的一个经常出现的智力测验问题,你站在三个封闭的门前,其中一个门后有奖品.当然,奖品在哪个门后是完全随机的.当你选定一个门以后,你的朋友打开其余两扇门中的一扇空门,显示门后没有奖品.此时你可以有两种选择,保持原来的选择,或改选另一扇没有被打开的门.当你作出最后选择以后,如果打开的门后有奖品,这个奖品就归你.现在有三种策略: (a) 坚持原来的选择; (b) 改选另一扇没...

5 赞同 · 1 评论 · 3 天前

简介 评论 989 点我发弹幕

李永乐老师官方 147.8万粉丝 + 关注

决胜21点中的“三门问题”是怎么回事？应该如何提高中奖的概率？李永乐老师讲解蒙提霍尔问题（2018最新）

10.4万 630 2018-06-26 AV25648623 未经授权禁止转载

有同学说 他最近看了一部电影叫做决胜21点

这里有一个问题叫做三门问题

问我是怎么回事

今天我们就来讨论一下这个三门问题

这个三门问题 也称之为蒙提霍尔问题

为什么叫蒙提霍尔问题呢

是因为 他是上个世纪美国有一个电视节目

这个电视节目里面有这么一个问题

然后主持人叫蒙提霍尔

所以就叫蒙提霍尔问题

MATLAB code2.m

用计算机验证一下反常识的蒙提霍尔问题（三门问题）

问题描述：三扇门，其中一扇后面有奖品，观众在三扇门中选择一扇，然后主持人打开一扇空的门，并询问观众是否要修改答案，选择另一扇门。你觉的到奖品的概率有多大？这个问题反常识在于，多数人都认为当主持人排除掉一扇门后，剩下的两扇选中的概率为二分之一，但从数学上讲，如果观众不修改答案，选中的概率为三分之一，而修改答...

```
bin/gcc.exe
c:/mingw/bin/./libexec/gcc/mingw32/6.3.0/lt0-wrap
/arc/gcc-6.3.0/configure --build=x86_64-pc-linux-gnu
with-gmp/mingw --with-mpfr --with-mpc/mingw --with-
to-wind32-registry --with-arch=i586 --with-tune=gnu
i-c++-fortran,ada --with-pkgversion='MinGW.org GCC
ared --enable-threads --with-dwarf2 --disable-sjlj
runtime:libas --with-libiconv-prefix/mingw --with-
devel-debug --enable-libgomp --disable-libvty --enab
MinGW.org GCC 6.3.0-1)
```

$\frac{1}{3}$ $\frac{2}{3}$

.1 .2 .3

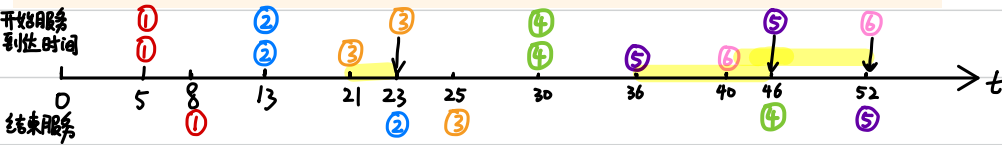
7 赞同 · 2 评论 · 2 个月前

(2) 模拟排队问题

假设某银行工作时间只有一个服务窗口，工作人员只能逐个的接待顾客。当来的顾客较多时，一部分顾客就需要排队等待。

假设：1) 顾客到来的间隔时间服从参数为0.1的指数分布 2) 每个顾客的服务时间服从均值为10，方差为4的正态分布(单位为分钟，若服务时间小于1分钟，则按1分钟计算) 3) 排队按先到先服务的规则，且不限队伍的长度，每天工作时长为8小时。

试回答下面的问题：1) 模拟一个工作日，在这个工作日共接待了多少客户，客户平均等待的时间为多少？ 2) 模拟100个工作日，计算出平均每日接待客户的个数以及每日客户的平均等待时长。



客户	①	②	③	④	⑤	⑥	...
到达时间	5	13	21	30	36	40	...
开始服务时间	5	13	23	30	46	52	...
结束服务时间	8	23	25	46	52	...	...

引入符号:

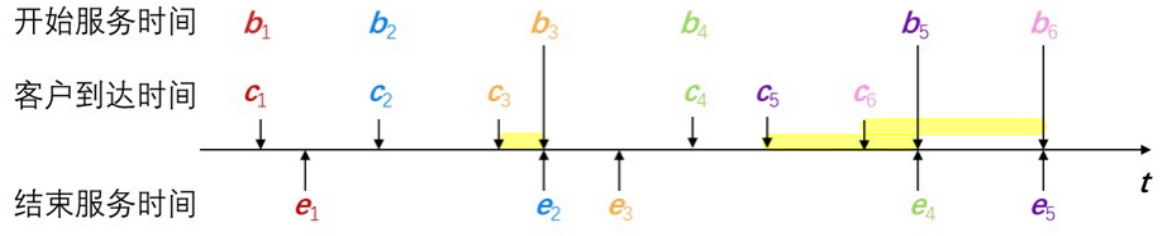
C_i : 第*i*个客户到达时间
 b_i : 第*i*个客户开始服务时间
 e_i : 第*i*个客户服务结束时间

题目中假设里面告诉我们的信息:

- ① 第*i*-1个客户和第*i*个客户到达的间隔时间 $x_i \sim E(0.1)$ 参数 $\lambda=0.1$ 的指数分布 (均值为10)
- ② 第*i*个客户的服务持续时间 $y_i \sim N(10, 4)$, 若 $y_i < 1 \Rightarrow$ 令 $y_i = 1$

概率密度函数

$$f(x_i) = \begin{cases} 0, & x_i < 0 \\ \lambda e^{-\lambda x_i}, & x_i \geq 0 \end{cases}$$



$$\begin{cases} C_i = C_{i-1} + x_i & (\text{初始值 } C_0 = 0) \\ e_i = b_i + y_i & (\text{初始值 } e_0 = 0) \\ b_i = \max(C_i, e_{i-1}) & (\text{初始值 } b_1 = C_1) \end{cases}$$

第*i*个客户开始服务的时间取决于该客户的到达时间和上一个客户服务结束的时间

取两者最大值



(3) 有约束的非线性规划问题

数学规划问题的分类:

$$\begin{cases} \min f(x) \rightarrow \text{目标函数} \\ \text{s.t. } g(x) \leq 0 \rightarrow \text{不等式约束} \\ h(x) = 0 \rightarrow \text{等式约束} \end{cases}$$

例: 求满足以下约束条件下函数 $f(x) = x_1 x_2 x_3$ 的最大值。

$$\text{s.t.} \begin{cases} -x_1 + 2x_2 + 2x_3 \geq 0 \\ x_1 + 2x_2 + 2x_3 \leq 72 \\ 10 \leq x_2 \leq 20 \\ x_1 - x_2 = 10 \end{cases}$$

- ☺ 若 $f(x), g_i(x), h_j(x)$ 为线性函数, 即为线性规划(LP);
- ☺ 若 $f(x), g_i(x), h_j(x)$ 至少一个为非线性, 即为非线性规划(NLP);
- ☺ 对于非线性规划, 若没有 $g_i(x), h_j(x)$, 即 $X=R^n$, 称为无约束非线性规划或无约束最优化问题; 否则称为约束非线性规划或约束最优化问题。

用蒙特卡罗模拟的思路:

- ① 从约束条件中求出每个变量受限的大致范围 ($a_i \leq x_i \leq b_i$ 且 $a_i, b_i \neq \infty$)
- ② 在上述范围中用随机数生成若干组试验点, 并验证它们是否满足所有的约束条件, 若满足, 则将其划分到可行组, 再从可行组中找到函数的最大值或最小值。

- ① 由 $x_1 - x_2 = 10$, 则 $x_2 = x_1 - 10$, 则 $f(x) = x_1(x_1 - 10)x_3$ 先消去变量 x_2
- ② $10 \leq x_2 \leq 20$ 则 $20 \leq x_1 \leq 30$ (由 $x_2 = x_1 - 10$ 可知)
- ③ 由 $-x_1 + 2x_2 + 2x_3 \geq 0$ 可知: $x_3 \geq \frac{x_1 - 2x_2}{2} \geq \frac{20 - 2 \times 20}{2} = -10$
- ④ 由 $x_1 + 2x_2 + 2x_3 \leq 72$ 可知: $x_3 \leq \frac{72 - x_1 - 2x_2}{2} \leq \frac{72 - 20 - 2 \times 10}{2} = 16$

所以我们可以生成一组随机数作为 x_1 , 再在 $[-10, 16]$ 中生成另一组随机数作为 x_3 , 例如:

x_1	x_2	x_3	是否满足条件	$f(x)$
25.6	15.6	-8.3	否	\
21.5	11.5	2.5	是	618.125
...	...	...	...	...

$$\begin{cases} -x_1 + 2x_2 + 2x_3 \geq 0 \\ x_1 + 2x_2 + 2x_3 \leq 72 \end{cases}$$

理论: 只要模拟的样本数足够多, 一定能找到最大值。

(后面规划问题是否还会再次讲到这个例题)



(4) 书店买书问题 (0-1规划)

某同学要从六家线上商城选购五本书籍 B1, B2, B3, B4, B5, 每本书籍在不同商家的售价以及每个商家的单次运费如下表所示, 请给该同学制定最省钱的选购方案。

(注: 在同一个店买多本书也只会收取一次运费)

	B1	B2	B3	B4	B5	运费
A 商城	18	39	29	48	59	10
B 商城	24	45	23	54	44	15
C 商城	22	45	23	53	53	15
D 商城	28	47	17	57	47	10
E 商城	24	42	24	47	59	10
F 商城	27	48	20	55	53	15

解: 设 $i=1, 2, \dots, 6$ 表示 A、B、...、E、F 六家商城, $j=1, 2, \dots, 5$ 表示 B1、B2、...、B5 五本书

记 m_{ij} 表示第 j 本书在第 i 家店的售价, q_i 表示第 i 家店的运费

引入 0-1 变量 $x_{ij} = \begin{cases} 1, & \text{该同学在第 } i \text{ 家店内购买第 } j \text{ 本书} \\ 0, & \text{该同学没有在第 } i \text{ 家店内购买第 } j \text{ 本书} \end{cases}$

那么有以下约束需要满足: $\sum_{i=1}^6 x_{ij} = 1, j=1, 2, \dots, 5$ (每本书只买一本)

我们的目标函数就是总花费, 其包含两个方面: (1) 五本书总价格 (2) 运费

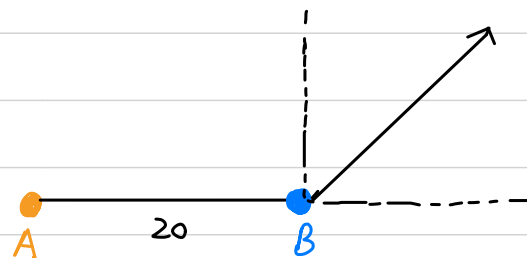
书价 = $\sum_{j=1}^5 \left[\sum_{i=1}^6 (x_{ij} \cdot m_{ij}) \right]$ 运费 = $\sum_{i=1}^6 (q_i \times t_i)$, 其中 $t_i = \begin{cases} 1, & \text{该同学在第 } i \text{ 家店消费} \\ 0, & \text{该同学没有在第 } i \text{ 家店消费} \end{cases}$
 $t_i = \max\{x_{i1}, x_{i2}, \dots, x_{i5}\}$



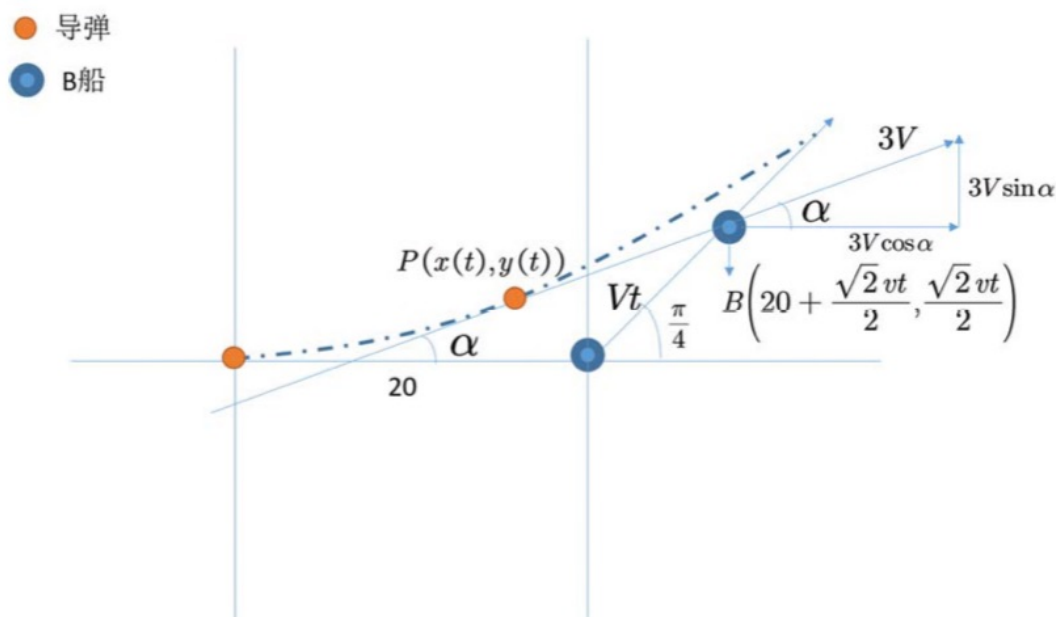
n 家店, m 本书, 所有可能情况有 n^m 种。
(后面在智能算法的部分会再次讲到这个例题)

(5) 导弹追踪问题

位于坐标原点的A船向位于其正东方20个单位的B船发射导弹，导弹始终对准B船，B船以时速V单位（常数）沿东北方向逃逸。若导弹的速度为3V，导弹的射程是50个单位，画出导弹运行的曲线，导弹是否能在射程内击中B船？



根据题意可作出如下示意图：



因为导弹始终对准B点，则在任一时刻，导弹行进的轨迹的切线要经过B船的位置。

如上图所示：经过时间 $t(t > 0)$ 时，B船的坐标是 $B\left(20 + \frac{\sqrt{2}Vt}{2}, \frac{\sqrt{2}Vt}{2}\right)$

假设导弹运行的轨迹上的任一点的坐标为 $P(x(t), y(t))$ ，且在P点的切线与x轴的夹角为 $\alpha(t)$ ，简记为 α 。因为导弹的速度方向可分解为水平方向和垂直方向，如图所示：水平方向的速度大小为 $3V \cos \alpha$ ，垂直方向的速度大小为 $3V \sin \alpha$ 。且满足以下等式关系：

$$\text{分解} \quad \frac{dx}{dt} = 3V \cos \alpha \quad (0.1)$$

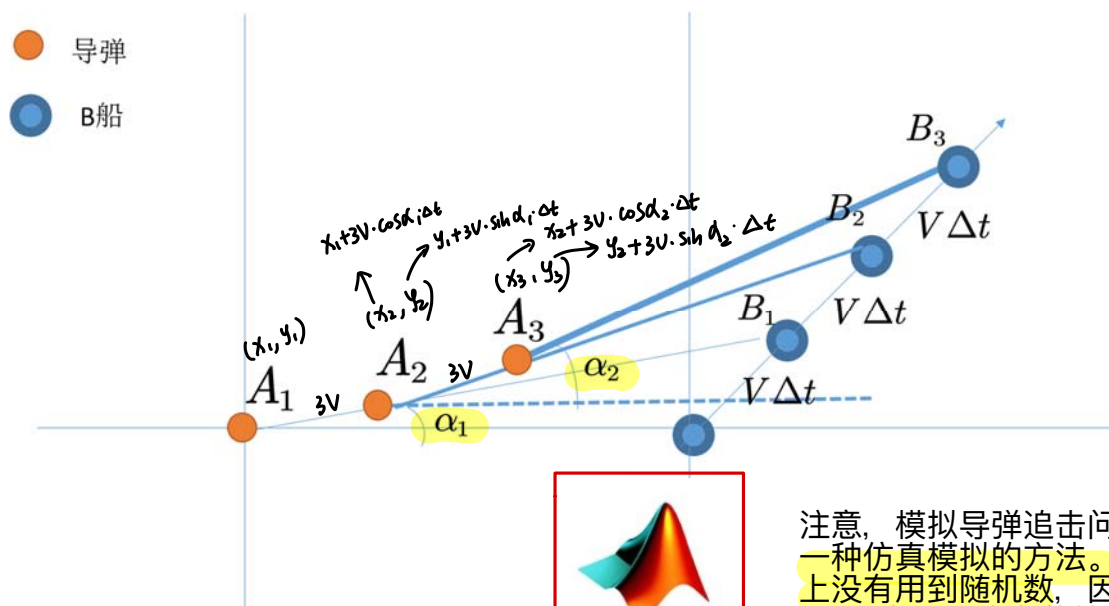
$$\frac{dy}{dt} = 3V \sin \alpha \quad (0.2)$$

下面我们建立以下的近似模型，来模拟导弹追击B船的过程。

首先将时间 T 划分为 n 个间隔，每一个间隔的时间都是 Δt ，为了让模拟效果尽量好，我们选取的 Δt 越小越好，这里我们不妨取 $\Delta t = 0.0000001$ 。

如下图所示，假设B船先走 Δt 个时间的距离到达 B_1 后，导弹才开始从 A_1 发射，那么下一个时间 Δt 内，导弹的方向一定是沿着直线 A_1B_1 ，假设该直线与水平方向夹角为 α_1 ，那么导弹在第二个 Δt 结束后到达 A_2 ，而此时B船到达 B_2 的位置，因为 B_1 坐标已知，那么很容易计算出直线 A_1B_1 的斜率 $\tan \alpha_1$ ，可反解求出 α_1 的大小，又因为每一个时间间隔 Δt 导弹飞行的距离为 $3V \Delta t$ ，即 $|A_i, A_{i+1}| = 3V \Delta t$ ，所以我们可以求出 A_2 的坐标，在下一个时间段，我们同理可求出 α_2 的大小，因此也可求出 A_3 的坐标，依次类推，直到最后导弹击中B船或者导弹达到自己的射程不能击中。

离散化

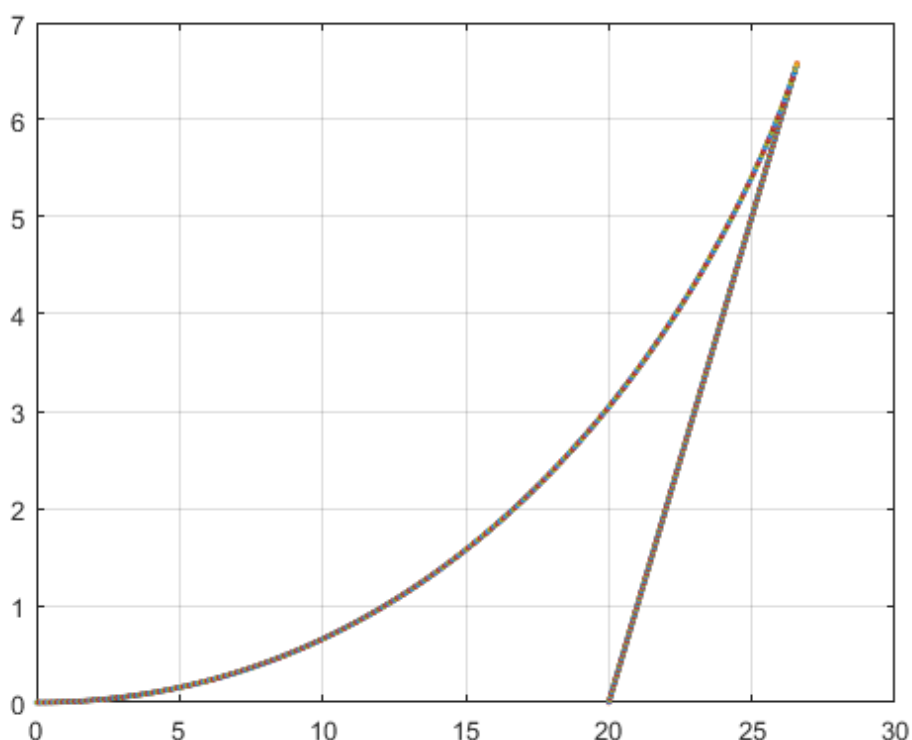


注意，模拟导弹追击问题更像是一种仿真模拟的方法。这里本质上没有用到随机数，因此严格意义上不能称为蒙特卡罗。

下面我们用 matlab 编程实现这一过程：
最后得到以下结论：

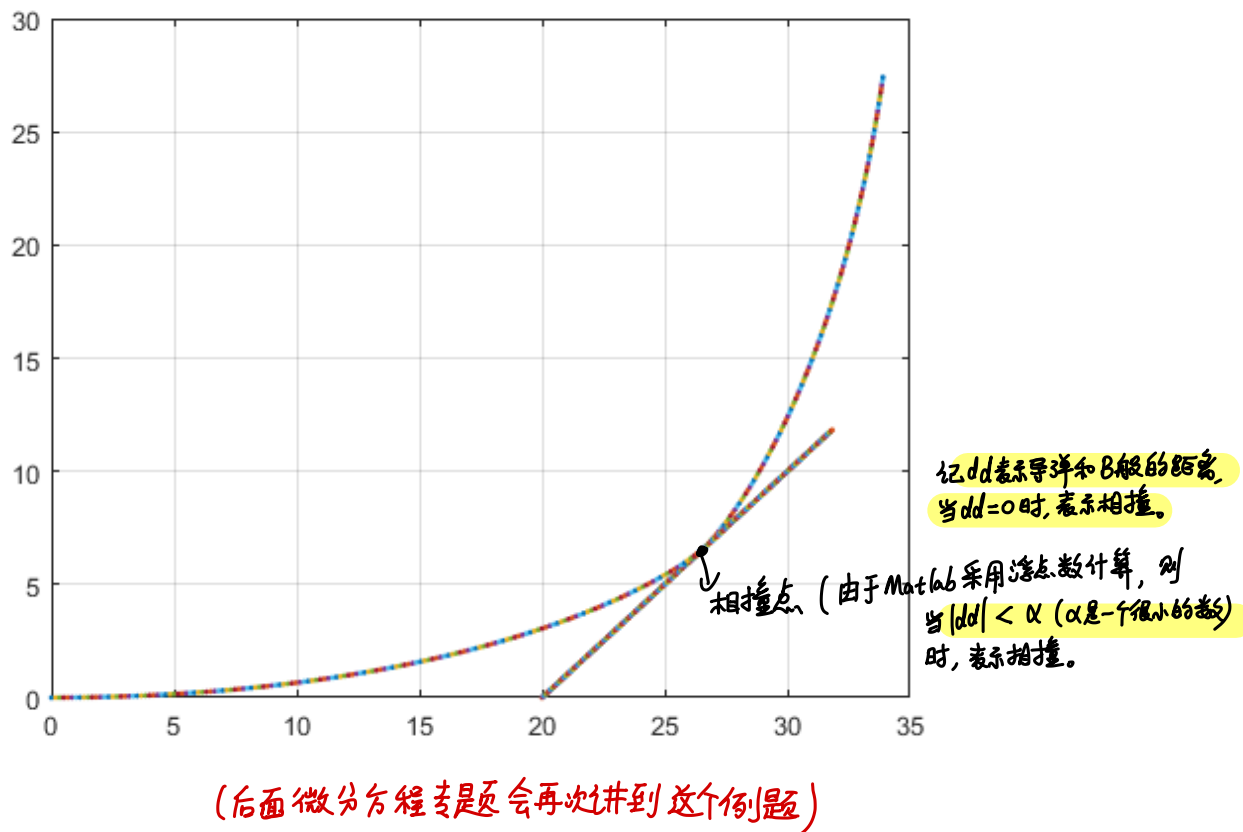
导弹一定能够击中 B 船，且击中 B 船的位置坐标是：(26.5532,6.5532)，导弹飞行的距离是 27.8032 个单位，它们与 B 船的速度 V 无关，速度 V 的大小仅仅会影响击中的时间。

效果图如下所示：



用代码模拟追击过程有以下要注意的点，原则上来说，时间间隔 $\Delta t \rightarrow 0$ 时，模拟效果最好，因此在代码中，我们定义的时间间隔越小越好。但由于计算机计算能力受限，间隔越短循环次数越多，运行速度较慢，因此我们可根据实际需要取较合适大小的时间间隔 Δt ；然而如果选取的 Δt 较大，会导致我们

判断导弹是否击中 B 船的判断条件受到限制，因为当 $\Delta t \rightarrow 0$ 时，如果导弹能击中 B 船，那么最终导弹和 B 船的距离也会趋于 0，但是因为我们 Δt 较大，所以最后两者可能相撞了，但是模拟出来相撞时的两者距离也较大，而如果我们选取判断条件中要求两者距离较小，就会导致误判，得出错误的结论。下图就是由于判断条件选择不合适，导致出现了错误判断的轨迹图：



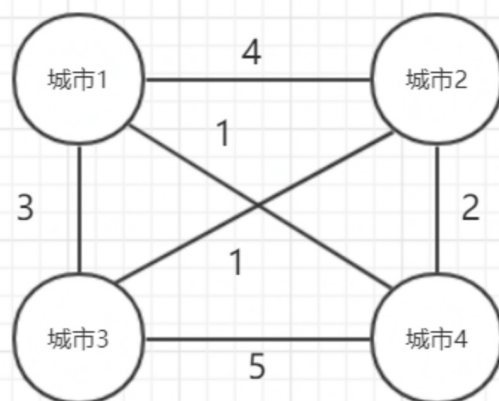
(6) 旅行商问题 (Traveling Salesman Problem, TSP)

图论

一个售货员必须访问 n 个城市，这 n 个城市是一个完全图，售货员需要恰好访问所有城市的一次，并且回到最终的城市。

城市于城市之间有一个旅行费用，售货员希望旅行费用之和最少。

完全图：完全图是一个简单的无向图，其中每对不同的顶点之间都恰连有一条边相连。



最小费用 城市1->城市3->城市2->城市4->城市1

假设有 n 个城市，则运算的开支为 $n!$

$n=10 \rightarrow 10^6$
 $n=15 \rightarrow 10^{12}$
 $n=20 \rightarrow 10^{18}$
 $(112 = 10^8)$

有 $(n-1)!$ 种路径
 每个路径要做 n 个求和



未来学习智能算法时会解决 n 很大的情形。

布丰用投针实验估计了 π 值，那么用什么简单方法可以估计自然对数的底数 e 的值？  修改

 修改

关注问题

 写回答

 邀请回答

 2 条评论

 分享

 举报

...

查看全部 17 个回答

 李恩志 
物理系博士

201 人赞同了该回答

一群人每人写一张卡片，卡片上是自己的名字。把卡片收上去，打乱次序，再随机地发给每一个人。每个人拿到的都不是自己卡片的概率趋近于 $\frac{1}{e}$ 。多做几次这个实验，用频率代替概率，求倒数，就可以了。跟布丰投针实验估算 π 挺像的。放上推导过程。

设有 n 个人，每人拿到的都不是自己卡片的情况总共有 a_n 种可能性， a_n 满足这样的递推公式，

$$a_n = (n - 1)(a_{n-1} + a_{n-2}), \quad n \geq 3, a_1 = 0, a_2 = 1.$$

于是每个人拿到的都不是自己的卡片的概率是 $p_n = \frac{a_n}{n!}$ ， p_n 满足这样的递推公式，

$$p_n - p_{n-1} = -\frac{1}{n}(p_{n-1} - p_{n-2})$$

很容易得到

$$p_n - p_{n-1} = \frac{(-1)^n}{n!}.$$

可以把 $p_n, n \geq 2$ 重新写成

$$p_n = \sum_{i=2}^n (p_i - p_{i-1}) + p_1$$

于是就有

$$p_n = \sum_{i=2}^n \frac{(-1)^i}{i!} = \sum_{i=0}^n \frac{(-1)^i}{i!}$$

取极限，得到

$$\lim_{n \rightarrow \infty} p_n = \sum_{n=0}^{\infty} \frac{(-1)^n}{n!} = \frac{1}{e}.$$

证明完毕。

这个题实际上是伯努利错装信封问题

作业1: 请你使用蒙特卡罗的方法对这个问题进行模拟，并估计出自然常数e的值

本节的作业讲解视频可在b站直接观看
搜索：清风数学建模作业讲解的视频
<https://www.bilibili.com/video/BV1i7411k7fB>

第2题

现在有一把神器，初始为 1 级，可免费领取（即价值为 0），可花费金币对其升级，每次 10000 金币，最多升到 5 级。



升级的成功率如下：

等级	1 级	2 级	3 级	4 级	5 级
1	65%	20%	10%	5%	0%
2	25%	40%	20%	10%	5%
3	10%	20%	40%	20%	10%
4	0%	10%	30%	40%	20%

以第一行和第三行为例，表格的解释：

1 级武器强化时，有 20%概率升到 2 级，10%概率升到 3 级，5%概率升到 4 级，65%概率不变。

3 级武器强化时，10%概率跌到 1 级，20%概率跌到 2 级，20%概率升到 4 级，10%概率升到 5 级。

问题：用蒙特卡罗模拟：5 级神器价值多少金币？（即打造一把 5 级的神器平均需要花费多少金币）

作业 3:

使用蒙特卡罗模拟求解下面的问题非线性规划问题

$$\min f(x_1, x_2) = 2x_1^2 + x_2^2 - x_1x_2 - 8x_1 - 3x_2$$

$$s.t. \begin{cases} 3x_1 + x_2 > 9 \\ x_1 + 2x_2 < 16 \\ x_1 > 0 \quad x_2 > 0 \end{cases}$$

作业 4. 某设备上安装有四只型号规格完全相同的电子管, 已知电子管寿命为 1 000 h ~ 2 000 h 之间的均匀分布. 当电子管损坏时有两种维修方案, 一是每次更换损坏的那一只; 二是当其中一只损坏时四只同时更换. 已知更换时间为换一只时需 1 h, 4 只同时换为 2 h. 更换时机器因停止运转每小时的损失为 20 元, 又每只电子管价格 10 元, 试用模拟方法决定哪一个方案经济合理?

(假定为整数)

投针实验代码

`rand(m,n)` 产生 $[0,1]$ 之间随机数 m 行 n 列 数组

$a + \text{rand}(m,n) * (b-a)$ 等价于 `unifrnd(a,b,m,n)`

`unifrnd(-2,2,3,2)` 在 $[-2,2]$ 之间 3行2列随机数

%% (2) 代码部分

```
l = 0.520; % 针的长度 (任意给的)
a = 1.314; % 平行线的宽度 (大于针的长度l即可)
n = 1000000; % 做n次投针试验, n越大求出来的pi越准确 (需要画图则n取小一点)
m = 0; % 记录针与平行线相交的次数
x = rand(1, n) * a / 2; % 在 $[0, a/2]$ 内服从均匀分布随机产生n个数, x中每一个元素表示针的中点和最近的一条平行线
phi = rand(1, n) * pi; % 在 $[0, \pi]$ 内服从均匀分布随机产生n个数, phi中的每一个元素表示针和最近的一条平行线的夹角
% axis([0,pi, 0,a/2]); box on; % 画一个坐标轴的框架, x轴位于0-pi, y轴位于0-a/2, 并打开图形的边框
for i=1:n % 开始循环, 依次看每根针是否和直线相交 从1到n循环
    if x(i) <= l / 2 * sin(phi(i)) % 如果针和直线相交
        m = m + 1; % 那么k就要加1
        % plot(phi(i), x(i), 'r.') % 模仿书上的那个图, 横坐标为phi, 纵坐标为x, 用红色的小点进行标记
        % hold on % 在原来的图形上继续绘制
    end
end
p = m / n; % 直线和平行线相交出现的频率
mypi = (2 * l) / (a * p); % 我们根据公式计算得到的pi
disp(['蒙特卡罗方法得到pi为: ', num2str(mypi)]) % 数字转为字符串
```

%% (3) 由于一次模拟的结果具有偶然性, 因此我们可以重复100次后再来求一个平均的pi

```
result = zeros(100,1); % 初始化保存100次结果的矩阵
l = 0.520; a = 1.314;
n = 1000000; m = 0; (每次需要重制)
for num = 1:100 (num = 重复100次) 为了求均值
    x = rand(1, n) * a / 2;
    phi = rand(1, n) * pi;
    for i=1:n
        if x(i) <= l / 2 * sin(phi(i))
            m = m + 1;
        end
    end
    p = m / n;
    mypi = (2 * l) / (a * p);
    result(num) = mypi; % 把求出来的mypi保存到结果矩阵中
end
mymeanpi = mean(result); % 计算result矩阵中保存的100次结果的均值
disp(['蒙特卡罗方法得到pi为: ', num2str(mymeanpi)])
```


2
%% 蒙特卡罗用于模拟三门问题

clear;clc

%% (1) 预备知识

% randi([a,b],m,n)函数可在指定区间[a,b]内随机取出大小为m*n的整数矩阵

randi([1,5],5,8) %在区间[1,5]内随机取出大小为5*8的整数矩阵

% 2 5 4 5 3 1 4 2

% 3 3 1 5 4 2 1 2

% 4 1 3 3 2 2 5 1

% 5 3 3 4 4 5 4 4

% 4 2 3 4 2 4 2 4

randi([1,5]) %在区间[1,5]内随机取出1个整数

% 3

randi([a,b],m,n)

strcat()

num2str()

% 字符串的连接方式: (1) ['字符串1','字符串2'] (2) strcat('字符串1','字符串2') (第一期视频第一讲)

['数学建模','学习交流']

strcat('数学建模','学习交流') 拼接

% num2str函数: 将数值转换为字符串 (第一期视频第一讲)

mystr = num2str(1224) % 注意观察工作区的mystr这个变量的值

disp([num2str(1224),'祝大家平安夜平平安安']) % disp函数是输出函数

%% (2) 代码部分

n = 100000; % n代表蒙特卡罗模拟重复次数

a = 0; % a表示不改变主意时能赢得汽车的次数

b = 0; % b表示改变主意时能赢得汽车的次数

for i = 1:n % 开始模拟n次

x = randi([1,3]); % 随机生成一个1-3之间的整数x表示汽车出现在第x扇门后

y = randi([1,3]); % 随机生成一个1-3之间的整数y表示自己选的门

% 下面分为两种情况讨论: $x=y$ 和 $x \neq y$ ($x \neq y$)

if x == y % 如果x和y相同, 那么我们只有不改变主意时才能赢

a = a + 1; b = b + 0;

else % $x \neq y$, 如果x和y不同, 那么我们只有改变主意时才能赢

a = a + 0; b = b + 1;

end

end

disp(['蒙特卡罗方法得到的不改变主意时的获奖概率为: ', num2str(a/n)]);

disp(['蒙特卡罗方法得到的改变主意时的获奖概率为: ', num2str(b/n)]);

change = randi([0, 1]) % change = 0 不改变主意, change = 1 改变主意

% 下面分为两种情况讨论: $x=y$ 和 $x \neq y$

if x == y % 如果x和y相同, 那么我们只有不改变主意时才能赢

if change == 0; % 不改变主意

a = a + 1; ✓

else % 改变了主意

c = c + 1; ✗

end

else % $x \neq y$, 如果x和y不同, 那么我们只有改变主意时才能赢

if change == 0 % 不改变主意

c = c + 1; ✗

else % 改变了主意

b = b + 1; ✓

end

end

end

disp(['蒙特卡罗方法得到的不改变主意时的获奖概率为: ', num2str(a/n)]);

disp(['蒙特卡罗方法得到的改变主意时的获奖概率为: ', num2str(b/n)]);

disp(['蒙特卡罗方法得到的失败的率为: ', num2str(c/n)]);

c=0
失败次数

%% 蒙特卡罗模拟排队问题

%% (1) 预备知识 (M, σ)

% normrnd(MU, SIGMA) 生成一个服从正态分布 (MU 参数代表均值, SIGMA 参数代表标准差) 的随机数

normrnd(0, 1)

% exprnd(M) 表示生成一个均值为 M 的指数分布随机数 (其对应的参数为 1/M)

exprnd(5) $\lambda = \frac{1}{M}$ $E(x) = \frac{1}{\lambda} = M \therefore \lambda = \frac{1}{M}$

% mean 函数是用来求解均值的函数 (第一期视频第五讲)

mean([1, 2, 3]) $E(x)$

% tic 函数和 toc 函数可以用来返回代码运行的时间, 例如我们要计算一段代码的运行时间, 就可以在这段代码前加上 tic, 在这段代码后加上

tic

a = 2^100

toc

} 时间经过 ...

toc

%% (2) 模型中出现的变量的说明

% x(i) 表示第 i-1 个客户和第 i 个客户到达的间隔时间, 服从参数为 0.1 的指数分布

% y(i) 表示第 i 个客户的服务持续时间, 服从均值为 10 方差为 4 (标准差为 2) 的正态分布 (若小于 1 则按 1 计算)

% c(i) 表示第 i 个客户的到达时间, 那么 $c(i) = c(i-1) + x(i)$, 初始值 $c_0 = 0$

% b(i) 表示第 i 个客户开始服务的时间

% e(i) 表示第 i 个客户结束服务的时间, 初始值 $e_0 = 0$

% 第 i 个客户结束服务的时间 = 第 i 个客户开始服务的时间 + 第 i 个客户的服务持续时间

% 即: $e(i) = b(i) + y(i)$

% 第 i 个客户开始服务的时间取决于该客户的到达时间和上一个客户结束服务的时间

% 即: $b(i) = \max(c(i), e(i-1))$, 初始值 $b_1 = c_1$;

% 第 i 个客户等待的时间 = 第 i 个客户开始服务的时间 - 第 i 个客户到达银行的时间

% 即: $wait(i) = b(i) - c(i)$

% w 表示所有客户等待时间的总和

% 假设一天内银行最终服务了 n 个顾客, 那么客户的平均等待时间 $t = w/n$

clear

tic % 计算 tic 和 toc 中间部分的代码的运行时间

i = 1; % i 表示第 i 个客户, 最开始取 i = 1

w = 0; % w 用来表示所有客户等待的总时间, 初始化为 0

e0 = 0; c0 = 0; % 初始化 e0 和 c0 为 0

x(1) = exprnd(10); % 第 0 个客户 (假想的) 和第 1 个客户到达的时间间隔

c(1) = c0 + x(1); % 第 1 个客户到达的时间

b(1) = c(1); % 第 1 个客户的开始服务的时间

while b(i) <= 480 % 开始设置循环, 只要第 i 个顾客开始服务的时间 (时刻) 小于 480, 就可以对其服务 (银行每天工作 8 小时, 转换为分钟就

y(i) = normrnd(10, 2); % 第 i 个客户的服务持续时间, 服从均值为 10 方差为 4 (标准差为 2) 的正态分布

if y(i) < 1 % 根据题目的意思: 若服务持续时间不足一分钟, 则按照一分钟计算

y(i) = 1;

end

e(i) = b(i) + y(i); % 第 i 个客户结束服务的时间 = 第 i 个客户开始服务的时间 + 第 i 个客户的服务持续时间

wait(i) = b(i) - c(i); % 第 i 个客户等待的时间 = 第 i 个客户开始服务的时间 - 第 i 个客户到达银行的时间

w = w + wait(i); % 更新所有客户等待的总时间

i = i + 1; % 增加一名新的客户

x(i) = exprnd(10); % 这位新客户和上一个客户到达的时间间隔

c(i) = c(i-1) + x(i); % 这位新客户到达银行的时间 = 上一个客户到达银行的时间 + 这位新客户和上一个客户到达的时间间隔

b(i) = max(c(i), e(i-1)); % 这个新客户开始服务的时间取决于其到达时间和上一个客户结束服务的时间

end

n = i - 1; % n 表示银行一天 8 小时一共服务的客户人数

t = w/n; % 客户的平均等待时间

disp(['银行一天 8 小时一共服务的客户人数为: ', num2str(n)])

disp(['客户的平均等待时间为: ', num2str(t)])

toc % 计算 tic 和 toc 中间部分的代码的运行时间

3 %% (4) 问题2的代码

```
clear
tic %计算tic和toc中间部分的代码的运行时间
day = 100; % 假设模拟100天
n = zeros(100,1); % 初始化用来保存每日接待客户数结果的矩阵
t = zeros(100,1); % 初始化用来保存每日客户平均等待时长的矩阵
for k = 1:day
    i = 1; % i表示第i个客户, 最开始取i=1
    w = 0; % w用来表示所有客户等待的总时间, 初始化为0
    e0 = 0; c0 = 0; % 初始化e0和c0为0
    x(1) = exprnd(10); % 第0个客户(假想的)和第1个客户到达的时间间隔
    c(1) = c0 + x(1); % 第1个客户到达的时间
    b(1) = c(1); % 第1个客户的开始服务的时间
    while b(i) <= 480 % 开始设置循环, 只要第i个顾客开始服务的时间(时刻)小于480, 就可以对其服
        y(i) = normrnd(10,2); % 第i个客户的服务持续时间, 服从均值为10方差为4(标准差为2)的正态分
        if y(i) < 1 % 根据题目的意思: 若服务持续时间不足一分钟, 则按照一分钟计算
            y(i) = 1;
        end
        e(i) = b(i) + y(i); % 第i个客户结束服务的时间 = 第i个客户开始服务的时间 + 第i个客户的服务持续时间
        wait(i) = b(i) - c(i); % 第i个客户等待的时间 = 第i个客户开始服务的时间 - 第i个客户到达银行的时间
        w = w + wait(i); % 更新所有客户等待的总时间
        i = i + 1; % 增加一名新的客户
        x(i) = exprnd(10); % 这位新客户和上一个客户到达的时间间隔
        c(i) = c(i-1) + x(i); % 这位新客户到达银行的时间 = 上一个客户到达银行的时间 + 这位新客户和上一个客户到达的时间间隔
        b(i) = max(c(i), e(i-1)); % 这个新客户开始服务的时间取决于其到达时间和上一个客户结束服务的时间
    end
    n(k) = i-1; % n(k)表示银行第k天服务的客户人数
    t(k) = w/n(k); % t(k)表示该银行第k天客户的平均等待时间
end
disp(['100个工作日内, 银行每日平均服务的客户人数为: ', num2str(mean(n))])
disp(['100个工作日内, 银行每日客户的平均等待时间为: ', num2str(mean(t))])
toc %计算tic和toc中间部分的代码的运行时间
```

求均值

4

%% (1) 预备知识

% (1) format long g 可以将Matlab的计算结果显示为一般的长数字格式 (默认会保留四位小数, 或使用科学计数法)

5/7 0.7142

5895*514100 3.0306e+09

format long g 保留完整结果

5/7 0.714285714285714...

5895*514100 3030619500

% (2) unifrnd(a,b,m,n)可以输出在[a,b]之间均匀分布的随机数组成的m行n列的矩阵。(等价于 $a + \text{rand}(m,n)*(b-a)$)

unifrnd(0,5,4,3)

% 4.07361843196589 3.16179623112705 4.78753417717149

% 4.5289596853781 0.487702024997048 4.82444267599638

% 0.63493408146753 1.39249109433524 0.788065408387741

% 4.5668792806951 2.73440759602492 4.85296390880308

%% (2) 代码部分

clc,clear;

tic %计算tic和toc中间部分的代码的运行时间

n=10000000; %生成的随机数组数

可以在初次求解后 缩小范围再求解

x1=unifrnd(20,30,n,1); % 生成在[20,30]之间均匀分布的随机数组成的n行1列的向量构成x1

x2=x1 - 10;

x3=unifrnd(-10,16,n,1); % 生成在[-10,16]之间均匀分布的随机数组成的n行1列的向量构成x3

fmax=-inf; % 初始化函数f的最大值为负无穷 (后续只要找到一个比它大的我们就对其更新)

for i=1:n

x = [x1(i), x2(i), x3(i)]; %构造x向量, 这里千万别写成了: $x = [x1, x2, x3]$

if (-x(1)+2*x(2)+2*x(3)>=0) & (x(1)+2*x(2)+2*x(3)<=72) % 判断是否满足条件

result = x(1)*x(2)*x(3); % 如果满足条件就计算函数值

if result > fmax % 如果这个函数值大于我们之前计算出来的最大值

∴求 $f(x)$ max

fmax = result; % 那么就更新这个函数值为新的最大值

X = x; % 并且将此时的x1 x2 x3保存到一个变量中

end

end

end

disp(strcat('蒙特卡罗模拟得到的最大值为',num2str(fmax)))

disp('最大值处x1 x2 x3的取值为:')

disp(X)

toc %计算tic和toc中间部分的代码的运行时间

约束条件复杂可用

%% 书店买书问题的蒙特卡罗的模拟

%% (1) 预备知识

% (1)unique函数: 剔除一个矩阵或者向量的重复值, 并将结果按照从小到大的顺序排列

% adj. 唯一的; 独一无二的 [ju'ni:k]

unique([1 2 5; 6 8 9; 2 4 6])

unique([5 6 8 8 4 1 6 2 2 4 8 4 5 6])

% (2)randi([a,b],m,n)函数可在指定区间[a,b]内随机取出大小为m*n的整数矩阵

randi([-5,5],2,6)

%% (2) 代码求解

min_money = +Inf; % 初始化最小的花费为无穷大, 后续只要找到比它小的就更新

min_result = randi([1, 6],1,5); % 初始化五本书都在第一家书店购买, 后续我们不断对其更新

%若min_result = [5 3 6 2 3], 则解释为: 第1本书在第5家店买, 第2本书在第3家店买, 第3本书在第6家店买, 第4本书在第2家店买, 第

n = 1000000; % 蒙特卡罗模拟的次数 (可更改)

M = [18 39 29 48 59

24 45 23 54 44

22 45 23 53 53

28 47 17 57 47

24 42 24 47 59

27 48 20 55 53]; % m ij 第j本书在第i家店的售价

freight = [10 15 15 10 10 15]; % 第i家店的运费

for k = 1:n % 开始循环

result = randi([1, 6],1,5); % 在1-6这些整数中随机抽取一个1*5的向量, 表示这五本书分别在哪家书店购买

index = unique(result); % 在哪些商店购买了商品, 因为我们等下要计算运费

money = sum(freight(index)); % 计算买书花费的运费

% 计算总花费: 刚刚计算出来的运费 + 五本书的售价

for i = 1:5

money = money + M(result(i),i);

end

if money < min_money % 判断刚刚随机生成的这组数据的花费是否小于最小花费, 如果小于的话

min_money = money % 我们更新最小的花费

min_result = result % 用这组数据更新最小花费的结果

end

end

min_money % 18+39+48+17+47+20

min_result I

打折 or 包邮 的更改 用if选择

6
%% 蒙特卡罗用于模拟导弹追击问题

%% (1) 预备知识

% mod(m,n)表示求m/n的余数

mod(8,3)

mod(1000,50)

% 设置横纵坐标的范围并标上字符

x = 1:0.01:3; (1-3, 间隔0.01)

y = x²; (对其中每个元素进行平方)

plot(x,y) % 画出x和y的图形

axis([0 3 0 10]) % 设置横坐标范围为[0, 3] 纵坐标范围为[0, 10]

pause(3) % 暂停3秒后再继续接下来的命令

text(2,4,'清风') % 在坐标为(2,4)的点上标上字符串: 清风

close % 关闭图形窗口

%% (2) 代码求解

% 1. 不画追击的示意图

clear;clc

v=200; % 任意给定B船的速度 (后期我们可以再改的)

dt=0.0000001; % 定义时间间隔

x=[0,20]; % 定义导弹和B船的横坐标分别为x(1)和x(2)

y=[0,0]; % 定义导弹和B船的纵坐标分别为y(1)和y(2)

t=0; % 初始化导弹击落B船的时间

d=0; % 初始化导弹飞行的距离

m=sqrt(2)/2; % 将sqrt(2)/2定义为一个常量, 使后面看起来很简洁

dd=sqrt((x(2)-x(1))^2+(y(2)-y(1))^2); % 导弹与B船的距离

while(dd>=0.001) % 只要两者的距离足够大, 就一直循环下去。(两者距离足够小时表示导弹击中, 这里的临界值要结合dt来取, 否则

t=t+dt; % 更新导弹击落B船的时间

d=d+3*v*dt; % 更新导弹飞行的距离

x(2)=20+t*v*m; y(2)=t*v*m; % 计算新的B船的位置 (注: m=sqrt(2)/2)

dd=sqrt((x(2)-x(1))^2+(y(2)-y(1))^2); % 更新导弹与B船的距离

tan_alpha=(y(2)-y(1))/(x(2)-x(1)); % 计算斜率, 即tan(alpha)

cos_alpha=sqrt(1/(1+tan_alpha^2)); % sec(alpha)^2 = (1+tan(alpha)^2)

sin_alpha=sqrt(1-cos_alpha^2); % sin(alpha)^2 + cos(alpha)^2 = 1

x(1)=x(1)+3*v*dt*cos_alpha; y(1)=y(1)+3*v*dt*sin_alpha; % 计算新的导弹的位置

if d>50 % 导弹的有效射程为50个单位

disp('导弹没有击中B船');

break; % 退出循环

end

if d<=50 & dd<0.001 % 导弹飞行的距离小于50个单位且导弹和B船的距离小于0.001 (表示击中)

disp(['导弹飞行',num2str(d),'单位后击中B船'])

disp(['导弹飞行的时间为',num2str(t*60),'分钟'])

end

end

6. 画图

```
t=0; % 初始化导弹击落B船的时间
d=0; % 初始化导弹飞行的距离
m=sqrt(2)/2; % 将sqrt(2)/2定义为一个常量, 使后面看起来很简洁
dd=sqrt((x(2)-x(1))^2+(y(2)-y(1))^2); % 导弹与B船的距离
for i=1:2
    plot(x(i),y(i),'k','MarkerSize',1); % 画出导弹和B船所在的坐标, 点的大小为1, 颜色为黑色(k), 用小点表示
    grid on; % 打开网格线
    hold on; % 不关闭图形, 继续画图
end
axis([0 30 0 10]) % 固定x轴的范围为0-30 估计y轴的范围为0-10
k = 0; % 引入一个变量 为了控制画图的速度 (因为Matlab中画图的速度超级慢)
while(dd >= 0.001) % 只要两者的距离足够大, 就一直循环下去。(两者距离足够小时表示导弹击中, 这里
    t=t+dt; % 更新导弹击落B船的时间
    d=d+3*v*dt; % 更新导弹飞行的距离
    x(2)=20+t*v*m; y(2)=t*v*m; % 计算新的B船的位置 (注: m=sqrt(2)/2)
    dd=sqrt((x(2)-x(1))^2+(y(2)-y(1))^2); % 更新导弹与B船的距离
    tan_alpha=(y(2)-y(1))/(x(2)-x(1)); % 计算斜率, 即tan(α)
    cos_alpha=sqrt(1/(1+tan_alpha^2)); % 利用公式: sec(α)^2 = (1+tan(α)^2) 计算出cos(α)
    sin_alpha=sqrt(1-cos_alpha^2); % 利用公式: sin(α)^2 + cos(α)^2 = 1 计算出sin(α)
    x(1)=x(1)+3*v*dt*cos_alpha; y(1)=y(1)+3*v*dt*sin_alpha; % 计算新的导弹的位置
    k = k + 1;
    if mod(k, 10) == 0 % 每刷新500次时间就画出下一个导弹和B船所在的坐标 mod(m,n)表示求m/n的余数
        for i=1:2
            plot(x(i),y(i),'k','MarkerSize',5);
            hold on; % 不关闭图形, 继续画图
        end
        pause(0.001); % 暂停0.001s后再继续下面的操作
    end
    if d > 50 % 导弹的有效射程为50个单位
        disp('导弹没有击中B船');
        break; % 退出循环
    end
    if d <= 50 & dd < 0.001 % 导弹飞行的距离小于50个单位且导弹和B船的距离小于0.001 (表示击中)
        disp(['导弹飞行', num2str(d), '个单位后击中B船'])
        disp(['导弹飞行的时间为', num2str(t*60), '分钟'])
    end
end
end
```


7

```
%% TSP(旅行商问题)
```

```
%% (1) 预备知识
```

```
plot([1,2],[5,10],'-o') % 画出一条线段, x范围是[1, 2], y范围是[5,10]
```

```
text(1.5,7.5,'清风') % 在坐标(1.5,7.5)处标上文本: 清风
```

```
randperm(5) % 生成1-5组成的一个随机序列(类似于洗牌的操作)
```

```
% 3 5 1 2 4
```

```
% 1 4 5 3 2
```

```
%% (2) 代码求解
```

```
clear;clc
```

```
% 只有10个城市的简单情况
```

```
coord = [0.6683 0.6195 0.4 0.2439 0.1707 0.2293 0.5171 0.8732 0.6878 0.8488;
```

```
0.2536 0.2634 0.4439 0.1463 0.2293 0.761 0.9414 0.6536 0.5219 0.3609]'; % 城市坐标矩阵, n行2列
```

```
% 38个城市, TSP数据集网站(http://www.tsp.gatech.edu/world/djtour.html) 上公测的最优结果6656。
```

```
% coord = [11003.611100,42102.500000;11108.611100,42373.888900;11133.333300,42885.833300;11155.833300,427
```

```
n = size(coord,1); % 城市的数目
```

```
figure(1) % 新建一个编号为1的图形窗口
```

```
plot(coord(:,1),coord(:,2),'o'); % 画出城市的分布散点图
```

```
for i = 1:n
```

```
text(coord(i,1)+0.01,coord(i,2)+0.01,num2str(i)) % 在图上标上城市的编号 (加上0.01表示把文字的标记往右上方偏移一点)
```

```
end
```

```
hold on % 等一下要接着在这个图形上画图的
```

```
d = zeros(n); % 初始化两个城市的距离矩阵全为0
```

权重连接矩阵

```
for i = 2:n
```

```
for j = 1:i
```

```
coord_i = coord(i,:); x_i = coord_i(1); y_i = coord_i(2); % 城市i的x坐标为x_i, y坐标为y_i
```

```
coord_j = coord(j,:); x_j = coord_j(1); y_j = coord_j(2); % 城市j的x坐标为x_j, y坐标为y_j
```

```
d(i,j) = sqrt((x_i-x_j)^2 + (y_i-y_j)^2); % 计算城市i和j的距离
```

```
end
```

```
end
```

```
d = d+d'; % 生成距离矩阵的对称的一面
```

```
min_result = +inf; % 假设最短的距离为min_result, 初始化为无穷大, 后面只要找到比它小的就对其更新
```

```
min_path = [1:n]; % 初始化最短的路径就是1-2-3-...-n
```

```
N = 1000000; % 蒙特卡罗模拟的次数
```

```
for i = 1:N % 开始循环
```

```
result = 0; % 初始化走过的路程为0
```

```
path = randperm(n); % 生成一个1-n的随机打乱的序列
```

```
for i = 1:n-1
```

```
result = d(path(i),path(i+1)) + result; % 按照这个序列不断的更新走过的路程这个值
```

```
end
```

```
result = d(path(1),path(n)) + result; % 别忘了加上从最后一个城市返回到最开始那个城市的距离
```

```
if result < min_result % 判断这次模拟走过的距离是否小于最小的距离, 如果小于就更新最短距离和最短的路径
```

```
min_path = path;
```

```
min_result = result
```

```
end
```

```
end
```


7

```
min_path
min_path = [min_path,min_path(1)]; % 在最短路径的最后面加上一个元素，即第一个点（我们要生成一个封闭的图形）
n = n+1; % 城市的个数加一个（紧随着上一步）
for i = 1:n-1
    j = i+1;
    coord_i = coord(min_path(i,:)); x_i = coord_i(1); y_i = coord_i(2);
    coord_j = coord(min_path(j,:)); x_j = coord_j(1); y_j = coord_j(2);
    plot([x_i,x_j],[y_i,y_j],'-') % 每两个点就作出一条线段，直到所有的城市都走完
    pause(0.5) % 暂停0.5s再画下一条线段
    hold on
end
```

先聚类 → 走路径？